

Smart Image Analyzer & Object Segmentation System

Computer Vision Project Report

Course: Computer Vision

Course Code: CSE3010

Student Name: Rashi Garg

Registration No.: 24BAI10740

Project Title: Smart Image Analyzer & Object Segmentation System

Academic Year: 2026

1. Introduction

Computer vision is an important field of artificial intelligence that enables computers to process and understand information from digital images. Various image processing techniques can be used to improve image quality, identify important features, separate regions, and analyze objects.

The Smart Image Analyzer & Object Segmentation System is a Python-based computer vision application developed to demonstrate multiple fundamental computer vision techniques through a single modular system.

The system accepts a single input image and processes it using image enhancement, edge and feature detection, image segmentation, and object analysis techniques. The generated results are saved as separate output images, along with a JSON report containing information about the processed image and detected objects.

The project uses Python along with OpenCV, NumPy, Matplotlib, and Scikit-learn. Pytest is used to perform automated testing of the major processing components.

The application follows a modular design in which different computer vision operations are implemented in separate Python modules. This makes the system easier to understand, test, maintain, and extend.

1.1 Objectives

The main objectives of the project are:

- To develop a modular computer vision application for analyzing a single image.
- To apply fundamental image enhancement techniques such as grayscale conversion, Gaussian filtering, histogram analysis, and histogram equalization.
- To detect important image features using Canny edge detection, Harris corner detection, and Hough line detection.
- To perform image segmentation using threshold-based segmentation and K-Means clustering.
- To analyze segmented regions using contour detection, bounding boxes, object area calculation, and object counting.
- To automatically generate a JSON report containing the analysis results.
- To demonstrate the practical implementation of computer vision concepts covered in the course.

2. Problem Statement

Digital images contain a large amount of visual information, but manually analyzing different image characteristics can be time-consuming. Different computer vision techniques are required to enhance images, detect important features, separate image regions, and analyze objects.

The problem addressed by this project is to develop a simple and modular computer vision system that can process an input image and apply multiple image analysis techniques in a single workflow.

The proposed system accepts a single input image and performs image enhancement, edge and feature detection, image segmentation, and object analysis. The results are saved as separate output images along with a JSON analysis report.

The system is designed to demonstrate the practical application of fundamental computer vision concepts while maintaining a modular and easy-to-understand implementation.

2.1 Scope of the Project

The project focuses on fundamental computer vision operations that can be applied to a single input image. The scope includes:

- Grayscale conversion
- Gaussian filtering
- Histogram generation
- Histogram equalization
- Canny edge detection
- Harris corner detection
- Hough line detection
- Threshold-based segmentation
- K-Means segmentation
- Contour detection
- Bounding box detection
- Object area calculation
- Object counting
- Automatic JSON report generation

The project does not focus on real-time video processing, deep learning-based object detection, or 3D computer vision.

2.2 Target Users

The system is primarily intended for:

- Computer vision students
- Beginners learning image processing
- Students experimenting with different computer vision techniques
- Users who want to observe the effect of multiple image processing methods on an image

3. Functional Requirements

FR1: Image Input

- The system shall allow the user to provide a single image as input through the input folder.

FR2: Image Validation

- The system shall verify that the input image exists and can be successfully read before processing.

FR3: Image Enhancement

- Grayscale conversion
- Gaussian filtering
- Histogram equalization
- Histogram generation

FR4: Edge and Feature Detection

- Canny edge detection
- Harris corner detection
- Hough line detection

FR5: Image Segmentation

- Threshold-based segmentation
- K-Means segmentation

FR6: Object Analysis

- Calculate object areas
- Generate bounding boxes
- Count detected objects
- Ignore regions with an area of 100 pixels or less

FR7: Output Generation

- The system shall save the processed images in the outputs folder.

FR8: Report Generation

- Image dimensions
- Number of detected objects
- Object areas
- Object locations
- Bounding box dimensions
- Techniques used

FR9: Error Handling

- The system shall display an appropriate error message when the input image is missing, invalid, or cannot be processed.

4. Non-Functional Requirements

NFR1: Usability

The system should be simple to execute and should provide clear status messages after processing.

NFR2: Maintainability

The implementation shall use separate modules for different computer vision operations so that individual components can be modified independently.

NFR3: Reliability

The system should validate the input image before performing processing operations.

NFR4: Error Handling

The system should handle invalid or missing input files without causing unexpected processing errors.

NFR5: Performance

The system should process a normal-sized input image within a reasonable amount of time on a standard computer.

NFR6: Resource Efficiency

The system should process one input image at a time and avoid unnecessary storage of intermediate data.

NFR7: Modularity

The system should separate image loading, enhancement, feature detection, segmentation, object analysis, and report generation into different modules.

NFR8: Testability

The major processing components should be testable independently using automated tests.

5. System Architecture

The Smart Image Analyzer & Object Segmentation System follows a modular architecture. The main controller coordinates separate modules responsible for image loading, enhancement, feature detection, segmentation, object analysis, and report generation.

The input image is processed through different computer vision operations, and the generated results are stored as output images and a JSON report.

5.1 Architecture Components

- Input Image — the user provides a single image through the input folder.
- Image Utilities (image_utils.py) — responsible for loading, validating, and saving images.
- Image Enhancement (enhancement.py) — grayscale conversion, Gaussian filtering, histogram generation, histogram equalization.
- Edge and Feature Detection (edge_detection.py) — Canny edge detection, Harris corner detection, Hough line detection.
- Image Segmentation (segmentation.py) — threshold-based segmentation, K-Means segmentation.
- Object Analysis (object_analysis.py) — contour detection, object area calculation, bounding box generation, object counting.
- Report Generation (report.py) — generates a JSON report containing image information and detected object measurements.
- Main Controller (main.py) — coordinates the different modules and controls the overall execution of the application.

5.2 Architecture Diagram

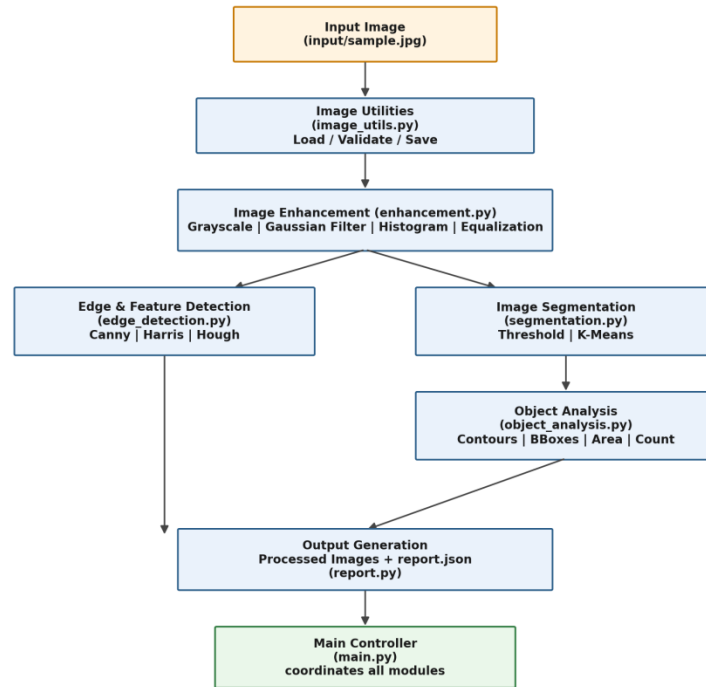


Fig. Smart Image Analyzer & Object Segmentation System
System Architecture Diagram

Fig. 5.1 — System Architecture Diagram

5.3 Technology Stack

Technology	Purpose
Python	Main programming language
OpenCV	Image processing and computer vision
NumPy	Numerical and image-array operations
Matplotlib	Histogram generation
Scikit-learn	K-Means clustering
Pytest	Automated testing

5.4 Project Module Structure

Module	Responsibility
main.py	Controls the complete workflow
image_utils.py	Image loading, validation and saving
enhancement.py	Image enhancement operations
edge_detection.py	Edge and feature detection

Module	Responsibility
segmentation.py	Image segmentation
object_analysis.py	Object and contour analysis
report.py	JSON report generation
test_project.py	Automated testing

6. Design Diagrams

This section presents the design diagrams that describe the structure and behaviour of the Smart Image Analyzer & Object Segmentation System.

6.1 Use Case Diagram

The use case diagram below shows the interactions between the User and the main functions offered by the system.

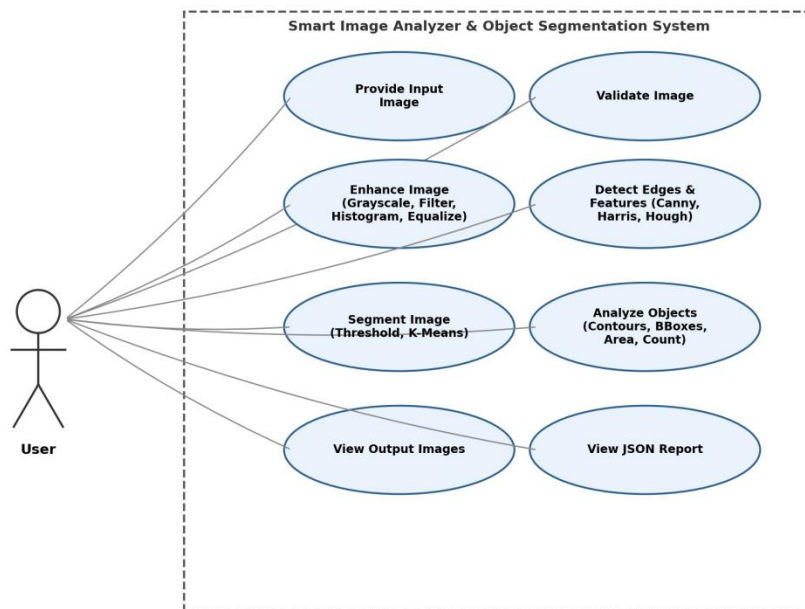


Fig. Use Case Diagram

Fig. 6.1 — Use Case Diagram

6.2 Process Flow / Workflow Diagram

The workflow diagram illustrates the sequential steps followed by the system, from loading the input image to generating the final JSON report.

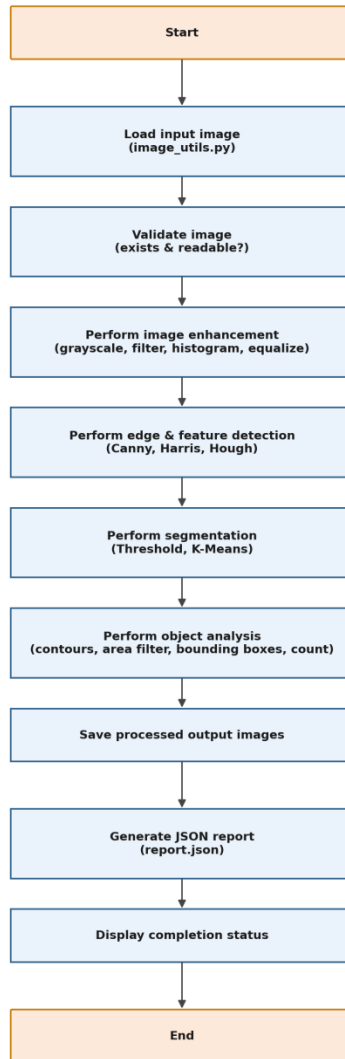


Fig. Process Flow / Workflow Diagram

Fig. 6.2 — Process Flow / Workflow Diagram

6.3 Sequence Diagram

The sequence diagram shows the order of interactions between the Main Controller and the individual processing modules during a single image-processing run.

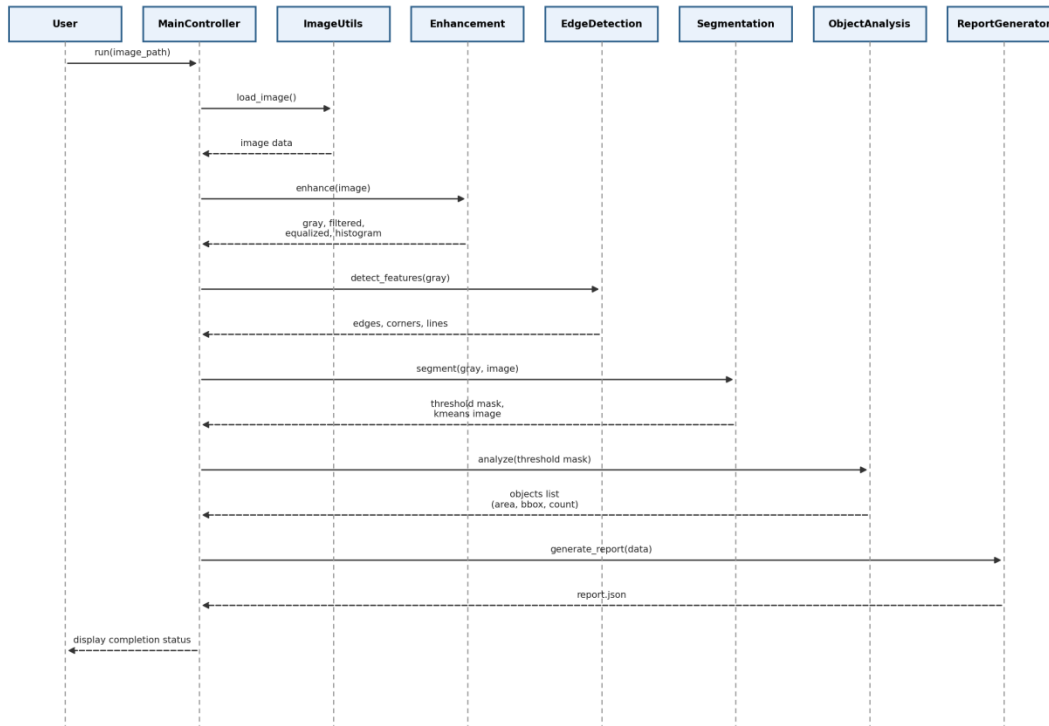


Fig. Sequence Diagram — single image processing run

Fig. 6.3 — Sequence Diagram for a single processing run

6.4 Class / Component Diagram

The class/component diagram shows the major modules of the system, their key methods, and the dependencies between them.

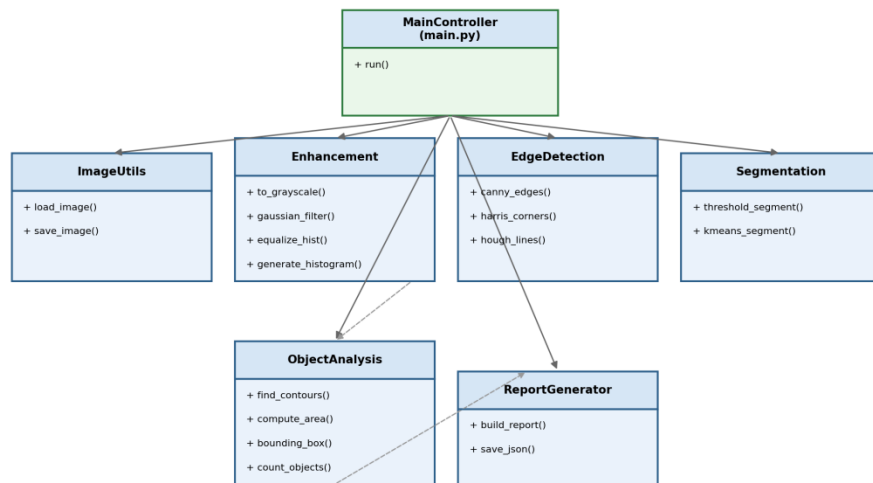


Fig. Class / Component Diagram
 (solid arrows: MainController coordinates each module;
 dashed arrows: data dependency between modules)

Fig. 6.4 — Class / Component Diagram

6.5 Database / Storage Design (ER Diagram)

The Smart Image Analyzer & Object Segmentation System does not use a relational database. Input and output data are handled as image files (JPG/PNG) and a JSON report, so no Entity–Relationship diagram or schema design applies to this project.

7. Design Decisions & Rationale

7.1 Selecting Appropriate Computer Vision Techniques

The project required combining multiple computer vision concepts into one meaningful workflow. To keep the project manageable while still demonstrating sufficient technical depth, the implementation focused on image enhancement, edge and feature detection, segmentation, and object analysis.

7.2 Handling Different Image Representations

Different computer vision operations require different image representations. For example, Canny edge detection, Harris corner detection, histogram equalization, and threshold segmentation operate on grayscale images, while K-Means segmentation is performed using colour pixel information. The project therefore includes separate processing steps for grayscale and colour images.

7.3 Choosing Segmentation Parameters

Threshold segmentation requires selecting an appropriate threshold value. A threshold of 127 (the midpoint of the 0–255 intensity range) was selected for the implemented sample workflow so that pixels are split evenly around the middle of the intensity range.

Similarly, K-Means segmentation requires selecting the number of clusters. The project uses three clusters ($K = 3$) to group pixels based on their colour characteristics, which is sufficient to separate a typical image into a small number of visually distinct regions without excessive computation. These values were kept fixed to maintain a simple and consistent workflow.

7.4 Object Filtering Threshold

Contour detection can identify many regions, including small regions that may not represent meaningful objects, such as noise or minor segmentation artefacts. To reduce the effect of such regions, the implementation ignores contours with an area of 100 pixels or less, so that only sufficiently large, meaningful regions are counted and reported as objects.

7.5 Modular Design Rationale

Implementing all computer vision operations in a single Python file would make the project difficult to understand, test, and maintain. The implementation was therefore divided into separate modules for image utilities, image enhancement, edge and feature detection, image segmentation, object analysis, and report generation, with `main.py` responsible for coordinating these modules. This separation supports the Modularity and Maintainability non-functional requirements described in Section 4.

8. Implementation Details

The Smart Image Analyzer & Object Segmentation System was implemented using Python. OpenCV is used for the main image processing operations, while NumPy, Matplotlib, and Scikit-learn support numerical processing, visualization, and clustering.

The implementation is divided into multiple Python modules. Each module performs a specific task, while main.py coordinates the complete workflow.

8.1 Image Loading and Validation

The input image is loaded using OpenCV. Before processing begins, the system checks whether the image file exists and whether OpenCV can successfully read it. If the image is missing or invalid, the system displays an error message and stops execution. The main functions used are:

```
load_image()  
save_image()
```

8.2 Image Enhancement Implementation

Grayscale Conversion

The colour image is converted into grayscale using OpenCV's colour conversion functionality.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Gaussian Filtering

Gaussian filtering is applied using a 5 x 5 kernel.

```
filtered = cv2.GaussianBlur(image, (5, 5), 0)
```

Histogram Equalization

Histogram equalization is applied to the grayscale image.

```
equalized = cv2.equalizeHist(gray_image)
```

Histogram Generation

Matplotlib is used to generate a histogram representing grayscale pixel intensity distribution.

8.3 Edge and Feature Detection Implementation

Canny Edge Detection

The Canny algorithm is applied to the grayscale image using threshold values of 100 and 200.

```
edges = cv2.Canny(gray_image, 100, 200)
```

Harris Corner Detection

The Harris corner detector is applied to identify corner features. The image is converted to floating-point representation before applying the detector.

Hough Line Detection

Probabilistic Hough Line Transform is applied to the Canny edge image to detect straight line segments. The detected lines are drawn on a copy of the original image.

8.4 Image Segmentation Implementation

Threshold Segmentation

A fixed threshold of 127 is applied to the grayscale image, producing a binary image containing foreground and background regions.

```
_, segmented = cv2.threshold(  
    gray_image,  
    127,  
    255,  
    cv2.THRESH_BINARY  
)
```

K-Means Segmentation

The colour pixels are reshaped into a two-dimensional array before clustering. The implementation uses three clusters.

```
kmeans = KMeans(  
    n_clusters=3,  
    random_state=42,  
    n_init=10  
)
```

After clustering, the cluster centers are used to reconstruct the segmented image.

8.5 Object Analysis Implementation

Object analysis is performed using contours obtained from the threshold-segmented image. The system calculates the area of each contour and ignores regions with an area of 100 pixels or less. For the remaining regions, bounding rectangles are calculated and drawn on the original image.

The system also stores the following information for each detected object:

- Area
- X-coordinate
- Y-coordinate
- Width
- Height

For the current sample image, the system detected 15 objects.

8.6 JSON Report Implementation

After object analysis is completed, the system generates a JSON report. The report contains image dimensions, channel information, detected object count, object measurements, and the techniques used by the application. The report is saved as outputs/report.json.

8.7 Output Generation

The system saves the results of the different processing operations as separate image files. The generated outputs include:

- grayscale.jpg
- filtered.jpg
- equalized.jpg

- histogram.png
- canny_edges.jpg
- harris_corners.jpg
- hough_lines.jpg
- threshold_segmentation.jpg
- kmeans_segmentation.jpg
- detected_objects.jpg
- report.json

8.8 Main Execution Flow

The main.py file coordinates the complete implementation. The execution flow is:

1. Load and validate the input image.
2. Perform image enhancement operations.
3. Perform edge and feature detection.
4. Perform threshold and K-Means segmentation.
5. Analyze objects from the threshold segmentation.
6. Save the processed images.
7. Generate the JSON report.
8. Display the completion status.

This modular implementation allows individual computer vision techniques to be tested and modified independently.

9. Screenshots / Results

The developed system was tested using a sample input image. The complete processing pipeline successfully generated outputs for image enhancement, edge and feature detection, segmentation, and object analysis. The system generated 10 processed image outputs and one JSON analysis report.

9.1 Input Image

The system accepts a single image placed in the input folder. The sample input used for testing was: input/sample.jpg

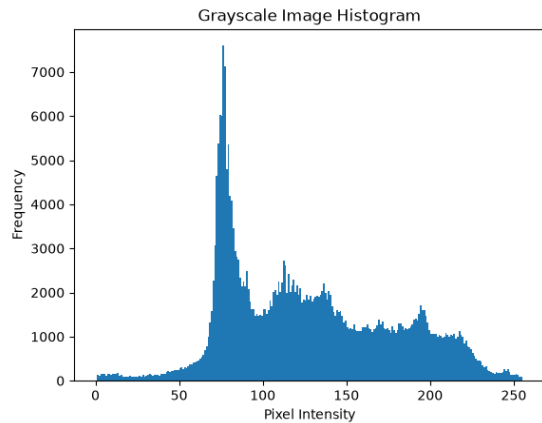
9.2 Grayscale Conversion

The original colour image was converted into a grayscale image. Grayscale conversion reduces the image from three colour channels to a single intensity channel and provides a suitable representation for several later processing operations such as Canny edge detection, Harris corner detection, histogram analysis, and threshold segmentation.



9.3 Histogram Analysis

A grayscale histogram was generated to show the distribution of pixel intensity values in the image.

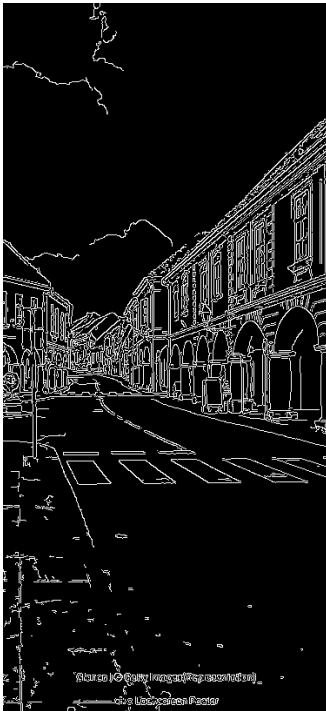


9.4 Image Filtering and Histogram Equalization

Gaussian filtering was applied to reduce noise and smooth the image. Histogram equalization was also performed on the grayscale image to improve the distribution of pixel intensities. The generated outputs are `filtered.jpg` and `equalized.jpg`, demonstrating the effect of image enhancement techniques on the input image.

9.5 Canny Edge Detection

Canny edge detection was applied to identify significant intensity changes in the image. The resulting edge image highlights boundaries and structural details present in the input image.



9.6 Harris Corner Detection

Harris corner detection was used to identify important corner features in the image. The detected corner points are highlighted in the generated output.



9.7 Hough Line Detection

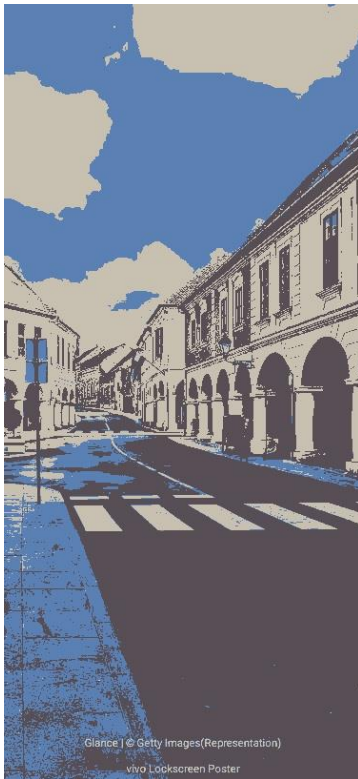
The Hough Line Transform was applied to the detected edge image to identify straight line segments. The detected lines are drawn over the original image. The generated output is `hough_lines.jpg`, which demonstrates how edge information can be further processed to identify geometric structures.

9.8 Threshold Segmentation

Threshold segmentation was applied to convert the grayscale image into a binary representation. Pixels below and above the selected threshold are separated into different regions. The generated output, `threshold_segmentation.jpg`, is subsequently used for contour-based object analysis.

9.9 K-Means Segmentation

K-Means clustering was applied to the colour pixels of the original image. The implementation uses three clusters ($K = 3$) to group pixels according to their colour characteristics.



9.10 Object Detection and Analysis

Contours were extracted from the threshold-segmented image. For each sufficiently large contour, the system calculates its area and bounding rectangle, and bounding boxes are then drawn around the detected regions.



For the sample input image, the system detected 15 objects. The detected object information is also stored in the generated JSON report.

9.11 Generated JSON Report

The system automatically generates outputs/report.json, containing: image width, image height, number of image channels, number of detected objects, area of each detected object, bounding box coordinates, bounding box dimensions, and the list of computer vision techniques used. The JSON report provides a structured representation of the analysis results and can be used for further processing.

9.12 Overall Result

The complete pipeline executed successfully and generated the expected outputs. The implementation demonstrates the use of multiple computer vision concepts in a single application, including image enhancement, feature detection, segmentation, contour analysis, and automated result reporting.

10. Testing Approach

Testing was performed to verify that the different modules of the Smart Image Analyzer & Object Segmentation System work correctly and produce valid outputs. The project uses the Pytest framework for automated testing. The test cases verify image loading, image processing, feature detection, segmentation, and object analysis.

10.1 Testing Approach Overview

The testing process focuses on checking whether:

- The input image exists and can be loaded.
- Image processing functions produce valid outputs.
- Output dimensions are correct.
- Different computer vision modules execute without errors.

- Segmentation produces the expected type of output.
- Object analysis returns a valid list of detected objects.

10.2 Test Cases

Test Case	Description	Expected Result
TC01	Check whether input image exists	Image file exists
TC02	Load input image	Image is loaded successfully
TC03	Convert image to grayscale	Two-dimensional grayscale image is produced
TC04	Apply Gaussian filtering	Filtered image has the same dimensions as input
TC05	Apply histogram equalization	Equalized image is produced successfully
TC06	Detect Canny edges	Two-dimensional edge image is produced
TC07	Detect Harris corners	Corner detection output has the original image dimensions
TC08	Detect Hough lines	Line detection output is produced successfully
TC09	Perform threshold segmentation	Binary segmented image is produced
TC10	Perform K-Means segmentation	Segmented image has the original image dimensions
TC11	Perform object analysis	Detected image and object list are returned

10.3 Automated Testing

The tests were executed using the following command:

```
pytest
```

The test suite completed successfully.

```
11 passed
```

This indicates that all 11 implemented test cases passed successfully during testing.

10.4 Functional Validation

In addition to automated testing, the complete application was executed using the sample input image. The application successfully:

9. Loaded the input image.
10. Generated grayscale, filtered, and equalized images.
11. Generated a grayscale histogram.
12. Detected edges, corners, and lines.

13. Performed threshold and K-Means segmentation.
14. Detected and analyzed objects.
15. Generated the JSON analysis report.

The sample execution detected 15 objects from the threshold-segmented image.

10.5 Error Handling Validation

The image loading module includes validation for missing and unreadable image files. If the specified image does not exist, the system raises a `FileNotFoundError`. If the image exists but cannot be read by OpenCV, the system raises a `ValueError`. The main program catches these errors and displays an appropriate error message instead of continuing with invalid input.

10.6 Testing Limitations

The current testing focuses mainly on functional correctness and output dimensions. The project does not calculate object-detection accuracy because a labeled ground-truth dataset was not used. The number of detected objects may also change depending on the input image and the selected segmentation threshold. Therefore, the reported result of 15 detected objects applies specifically to the sample input image used during testing.

11. Challenges Faced

During the development of the Smart Image Analyzer & Object Segmentation System, several implementation challenges were encountered. These challenges helped in understanding practical aspects of computer vision implementation and modular software development.

11.1 Combining Multiple Computer Vision Techniques

The project required combining multiple computer vision concepts into one meaningful workflow without making the application overly complex or difficult to evaluate.

11.2 Handling Different Image Representations

Different computer vision operations required different image representations — grayscale for most feature-detection and thresholding operations, and colour pixel data for K-Means segmentation — which required careful handling of image formats throughout the pipeline.

11.3 Object Detection from Segmented Images

Contour detection can identify many regions, including small regions that may not represent meaningful objects. Deciding on and applying a suitable minimum-area filter (100 pixels) required experimentation with the sample image.

11.4 Managing Multiple Output Files

The project produces several intermediate and final results. To keep the outputs organized, all generated processing results are stored in the outputs directory, while the source code is separately organized inside the src directory. This separation makes the project easier to understand and maintain.

11.5 Maintaining Modular Code

Implementing all computer vision operations in a single Python file would have made the project difficult to maintain. The implementation was therefore divided into separate modules for image utilities, image enhancement, edge and feature detection, image segmentation, object analysis, and report generation, coordinated by main.py.

11.6 Testing Individual Components

Another challenge was ensuring that the different processing functions produced valid outputs. Automated tests were created for the major modules using Pytest, and the final test execution resulted in 11 passed test cases, helping verify that individual components functioned correctly before considering the complete workflow.

12. Learnings & Key Takeaways

The development of the Smart Image Analyzer & Object Segmentation System provided practical experience with fundamental computer vision concepts as well as modular software development.

12.1 Understanding Image Processing

The project provided practical understanding of how digital images can be represented and processed using pixel intensity values and colour channels. The implementation demonstrated grayscale conversion, image filtering, histogram analysis, and histogram equalization.

12.2 Understanding Feature Detection

The project provided hands-on experience with different feature detection techniques. Canny edge detection was used to identify edges, Harris corner detection was used to identify corner features, and Hough Line Transform was used to detect straight line structures.

12.3 Understanding Image Segmentation

The project helped demonstrate how an image can be divided into meaningful regions. Both threshold-based segmentation and K-Means segmentation were implemented and their outputs were compared visually.

12.4 Understanding Contour-Based Object Analysis

The project demonstrated how contours can be extracted from a segmented image and used for object analysis. Bounding rectangles and contour areas were calculated to obtain basic information about detected regions.

12.5 Understanding Unsupervised Clustering

K-Means clustering was implemented for image segmentation, providing practical understanding of how pixels can be grouped into clusters based on their colour characteristics without requiring manually labeled training data.

12.6 Modular Software Development

The project demonstrated the importance of separating different functionalities into independent modules. The source code was organized into separate files for image utilities, enhancement, edge detection, segmentation, object analysis, and report generation.

12.7 Testing and Validation

The project provided experience with automated software testing using Pytest. A total of 11 test cases were implemented, covering the major processing functions, and all 11 tests passed successfully.

12.8 Documentation and Version Control

The project also provided practical experience in documenting a software project and maintaining its source code using Git and GitHub. The project includes a README file, project statement, technical documentation, test cases, and organized source code.

12.9 Overall Conclusion

Overall, the Smart Image Analyzer & Object Segmentation System successfully demonstrates the application of fundamental computer vision techniques — image enhancement, edge and feature detection, segmentation, and contour-based object analysis — in a single modular application. All 11 automated test cases passed successfully, and the project also demonstrates important software development practices such as modular code organization, error handling, automated testing, documentation, and Git-based version control.

13. Future Enhancements

The current system demonstrates a functional image analysis pipeline using fundamental computer vision techniques. The project can be further extended with additional features and more advanced methods.

13.1 Automatic Threshold Selection

The current implementation uses a fixed threshold value of 127. In the future, automatic thresholding methods such as Otsu's thresholding could be incorporated to select a suitable threshold based on the input image.

13.2 Support for Multiple Images

The current system processes one input image at a time. Future versions could allow users to provide multiple images and process them automatically in a batch.

13.3 Graphical User Interface

A graphical user interface could be added to make the system easier to use. Users could select an image through a file browser and view the different processing results directly within the application.

13.4 Advanced Object Detection

The current implementation uses contour-based analysis for detecting objects. Future versions could incorporate more advanced object detection techniques to identify and classify specific objects in an image.

13.5 Improved Segmentation

The segmentation module could be extended with additional techniques such as adaptive thresholding, watershed segmentation, or other clustering-based approaches, which could improve segmentation performance for images with varying lighting and complex backgrounds.

13.6 Quantitative Evaluation

A labeled dataset could be introduced in a future version to provide quantitative evaluation of segmentation and object detection results. Metrics such as precision, recall, Intersection over Union (IoU), or segmentation accuracy could then be calculated where appropriate.

13.7 Real-Time Video Processing

The system could be extended from static image processing to real-time video processing using a camera, allowing the implemented detection and segmentation techniques to operate continuously on video frames.

13.8 Improved Report Generation

The current JSON report contains basic image and object information. Future versions could generate a more detailed report containing processing statistics, execution time, comparisons between segmentation methods, and visual summaries.

14. References

1. OpenCV Documentation, Open Source Computer Vision Library. <https://docs.opencv.org/>
2. NumPy Documentation. <https://numpy.org/doc/>
3. Matplotlib Documentation. <https://matplotlib.org/stable/>
4. Scikit-learn Documentation, K-Means Clustering. <https://scikit-learn.org/stable/modules/clustering.html>
5. Pytest Documentation. <https://docs.pytest.org/>
6. VITyarthi — Build Your Own Project: Project Instructions and Evaluation Guidelines.
7. Computer Vision course materials, CSE3010.
8. Project GitHub Repository:
<https://github.com/RashiGarg727/SmartImageAnalyzer>